

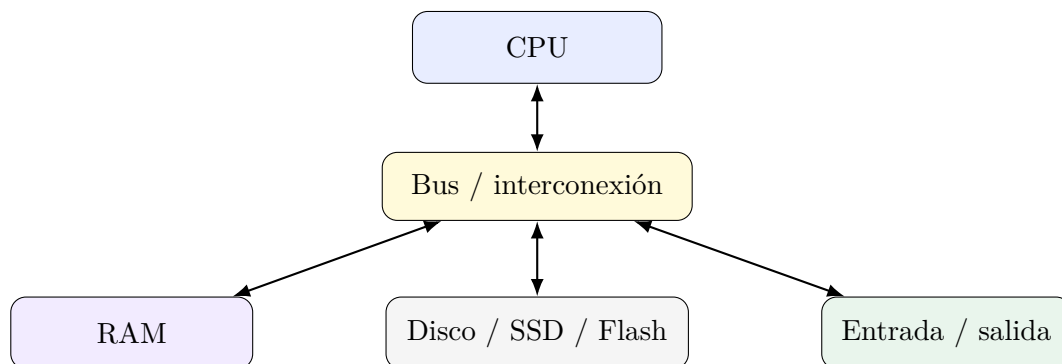
Arquitectura general y memoria del programa

1.1. Una vista simplificada del computador

De manera general, un computador puede entenderse como un sistema formado por varios elementos que trabajan de forma coordinada. Para los propósitos de este capítulo, conviene observar especialmente cuatro componentes: la unidad de procesamiento, la memoria principal, el almacenamiento no volátil y los dispositivos de entrada y salida.

La **CPU** ejecuta instrucciones. La **RAM** almacena temporalmente programas y datos mientras el sistema está encendido. El **almacenamiento no volátil**, como un disco, una unidad SSD o una memoria Flash, conserva información incluso cuando el equipo se apaga. Los dispositivos de **entrada y salida** permiten que el computador se comunique con el exterior.

- la CPU ejecuta instrucciones;
- la RAM almacena temporalmente programas y datos durante la ejecución;
- el almacenamiento no volátil conserva programas y archivos de forma permanente;
- los dispositivos de entrada y salida permiten comunicarse con el exterior;
- buses, controladores e interconexiones permiten mover datos, instrucciones y direcciones.



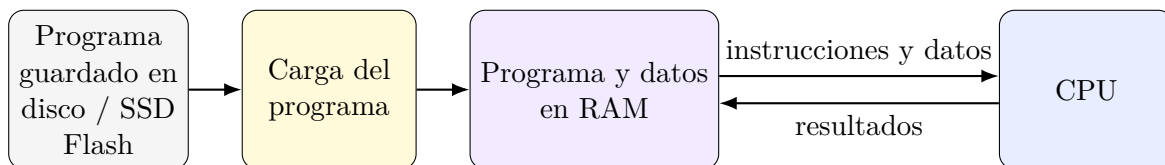
Idea central

Una variable existe en memoria. Por eso tiene sentido preguntar por su dirección.

1.2. Programa almacenado y programa en ejecución

Un programa puede estar guardado de forma permanente en un medio de almacenamiento no volátil. En un computador personal, ese almacenamiento suele ser un disco, una unidad SSD o una memoria Flash. En ese estado, el programa es simplemente información guardada: todavía no está ejecutándose.

Cuando el programa se ejecuta, el sistema operativo carga en la RAM las instrucciones y datos necesarios. A partir de allí, la CPU puede ir tomando instrucciones, procesando datos y actualizando valores. En la práctica también intervienen memorias caché internas del procesador, pero para una primera explicación basta con distinguir entre almacenamiento permanente y memoria de trabajo.



Memoria

El almacenamiento no volátil conserva el programa cuando el equipo está apagado. La RAM, en cambio, se usa como memoria de trabajo durante la ejecución.

Cuando un programa se ejecuta, sus instrucciones, sus variables, sus datos y otros elementos necesarios para el funcionamiento se organizan en memoria. Esa memoria puede entenderse como un conjunto de posiciones, y cada posición tiene una dirección.

1.3. La memoria RAM como espacio de trabajo

Memoria como conjunto de posiciones

Para visualizar la idea anterior, puede imaginarse la RAM como una colección ordenada de posiciones. Cada posición tiene una dirección y puede contener una instrucción, una variable, un dato temporal u otra información necesaria para la ejecución del programa.

En los diagramas de memoria es común escribir las direcciones en **base 16**, también llamada **sistema hexadecimal**. Por eso aparecen valores como 0x1000, 0x1004 o 0x2000. El prefijo 0x indica que el número está escrito en hexadecimal.

La Figura 1.1 presenta una representación conceptual de la RAM durante la ejecución de un programa. En la columna izquierda se muestran direcciones de memoria y en la columna derecha se muestra el contenido asociado con algunas de esas posiciones. Puede imaginarse esta memoria como un conjunto ordenado de casillas o cajones: cada casilla tiene una dirección y en ella puede guardarse una instrucción, una variable, un dato temporal u otro elemento necesario para la ejecución de una tarea.

Durante la ejecución, la CPU no lee esas posiciones de manera desordenada. En términos generales, va siguiendo una secuencia de instrucciones. Para ello, el procesador utiliza internamente un registro conocido como **contador de programa**, que permite llevar la referencia de la próxima instrucción que debe ejecutarse. Después de ejecutar una instrucción, normalmente se avanza hacia la siguiente, salvo que el programa indique otra ruta mediante una decisión, un ciclo o una llamada a función.

Así, la memoria puede verse como un espacio organizado por direcciones, mientras que la ejecución del programa puede entenderse como un recorrido ordenado por instrucciones. Esta idea ayuda a relacionar tres elementos importantes: las instrucciones que indican qué hacer, las direcciones que permiten ubicar información y los datos que se leen, se modifican o se almacenan.

Las direcciones que aparecen en la figura son simbólicas. No pretenden representar direcciones reales de un computador específico, sino ayudar a comprender que los elementos de un programa se ubican en memoria y pueden asociarse con posiciones identificables.

RAM durante la ejecución

Dirección	Contenido
0x1000	instrucción del programa
0x1004	otra instrucción del programa
0x2000	variable a con valor 10
0x2004	variable b con valor 20
0x2008	variable suma con valor 30
0x3000	dato temporal usado durante la ejecución

Figura 1.1: Representación conceptual de la memoria como un conjunto de posiciones con direcciones. Las direcciones están escritas en base 16 y son simbólicas; se usan solo con fines explicativos.

En la Figura 1.1, las primeras filas representan instrucciones del programa. Esto recuerda que, durante la ejecución, la memoria no solo contiene datos numéricos escritos por el programador, sino también instrucciones que la CPU debe interpretar y ejecutar.

Más abajo aparecen algunas variables: **a**, **b** y **suma**. En este ejemplo conceptual, la variable **a** tiene asociado el valor 10, la variable **b** tiene asociado el valor 20 y la variable **suma** tiene asociado el valor 30. Lo importante no es la dirección exacta usada en el dibujo, sino la relación entre una posición de memoria y el contenido que se guarda allí.

La última fila muestra un dato temporal usado durante la ejecución. Este tipo de dato representa información auxiliar que el programa puede necesitar mientras realiza operaciones. Por ejemplo, durante una suma, una comparación o una llamada a función, pueden existir valores intermedios que también deben ubicarse en memoria o en registros del procesador.

Esta figura permite insistir en una idea esencial: un programa en ejecución no está formado únicamente por instrucciones escritas en el código fuente. También involucra instrucciones cargadas en memoria, variables, valores temporales, datos de entrada, resultados parciales y otros elementos necesarios para que la ejecución pueda avanzar.

Relación entre instrucciones, direcciones y datos

Después de distinguir entre un programa almacenado y un programa en ejecución, conviene observar con más detalle qué ocurre cuando el programa ya está cargado en memoria. Durante la ejecución, no solo se manipulan datos. También se trabaja con instrucciones y direcciones.

Una **instrucción** indica qué operación debe realizarse. Una **dirección** permite ubicar una posición en memoria. Un **dato** es el contenido que se lee, se modifica o se almacena en alguna de esas posiciones.

La Figura 1.2 muestra esta relación mediante un ejemplo sencillo. Se usan instrucciones muy básicas de C para representar cómo algunas variables pueden ubicarse en memoria y cómo sus valores quedan asociados a ciertas direcciones. Las direcciones están escritas en base 16 y son simbólicas; no representan direcciones reales de un computador específico.

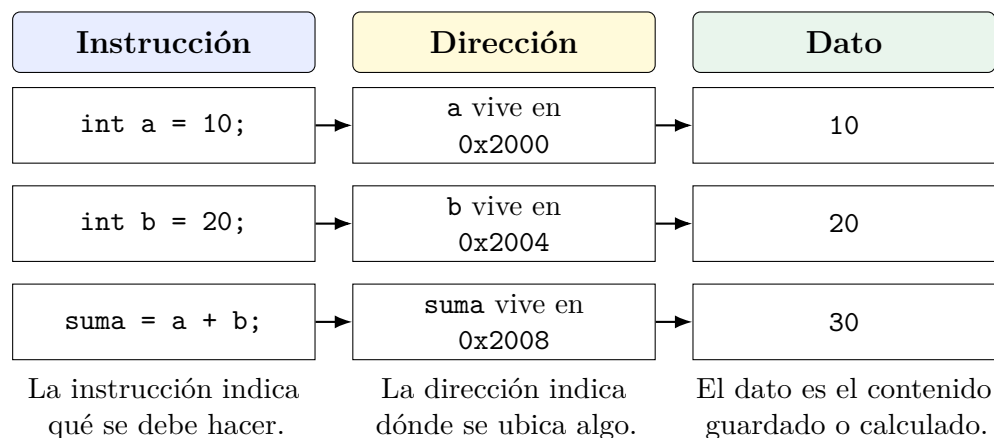


Figura 1.2: Relación conceptual entre instrucción, dirección y dato. Las direcciones 0x2000, 0x2004 y 0x2008 están escritas en base 16 y se usan solo con fines explicativos.

En la Figura 1.2, la primera fila representa la instrucción `int a = 10;`. Esta instrucción declara una variable llamada `a` y almacena en ella el valor 10. La dirección 0x2000 representa, de manera simbólica, el lugar de memoria asociado con esa variable.

La segunda fila representa la instrucción `int b = 20;`. De manera similar, la variable `b` queda asociada con otra posición de memoria, representada simbólicamente por la dirección 0x2004, y allí se almacena el valor 20.

La tercera fila representa la instrucción `suma = a + b;`. En este caso, el programa toma los valores almacenados en `a` y `b`, realiza la operación de suma y guarda el resultado en la variable `suma`. En la figura, esa variable se asocia simbólicamente con la dirección 0x2008, donde queda

almacenado el valor 30.

Esta representación permite relacionar el código fuente con la memoria durante la ejecución. Las instrucciones indican acciones; las direcciones permiten ubicar posiciones de memoria; y los datos corresponden a los valores que se almacenan, se leen o se modifican.

Durante la ejecución, la CPU sigue una secuencia de instrucciones. De forma general, puede decirse que el procesador mantiene una referencia hacia la próxima instrucción que debe ejecutar mediante un registro conocido como **contador de programa**. Después de ejecutar una instrucción, normalmente se pasa a la siguiente, salvo que el programa indique otro camino mediante una decisión, un ciclo o una llamada a función.

Clave de lectura

Un programa almacenado permanece guardado en un medio no volátil. Un programa en ejecución, en cambio, usa memoria de trabajo. En esa memoria se organizan instrucciones, variables, datos temporales y resultados parciales mediante posiciones identificables.

1.4. Modelo de ejecución en C

Continuando, conviene precisar cómo se organiza la ejecución de un programa escrito en C. En este documento se usará la expresión **modelo de ejecución** en un sentido didáctico: se refiere a la forma en que un programa inicia, ejecuta instrucciones, usa funciones, conserva datos en memoria y finaliza su trabajo.

En el lenguaje C, esta explicación se relaciona con nociones formales como el *entorno de ejecución*, el inicio del programa y la terminación del programa. La norma del lenguaje distingue, por ejemplo, entre entornos alojados y entornos independientes. En un **entorno alojado**, típico de un computador con sistema operativo, el programa inicia mediante una función llamada `main` [?]. En otros contextos, especialmente en sistemas embebidos o entornos simplificados, la organización visible para quien programa puede cambiar o quedar parcialmente oculta por el entorno de desarrollo.

Clave de lectura

La expresión **modelo de ejecución** ayuda a describir cómo se comporta un programa durante su funcionamiento: dónde inicia, cómo avanza, cómo usa funciones, cómo trabaja con datos en memoria y cómo finaliza.

En un programa C ejecutado en un **entorno alojado**, como ocurre normalmente en un computador con sistema operativo, el punto de entrada más conocido es la función `main`. A partir de ella se ejecutan instrucciones, se llaman funciones y se usan variables almacenadas en memoria.

En otros contextos, especialmente en sistemas embebidos, el punto de entrada puede estar organizado de otra manera. En Arduino, por ejemplo, quien programa escribe principalmente `setup()` y `loop()`, mientras el entorno se encarga de llamar esas funciones desde una estructura principal que no siempre se muestra al usuario.

La función main también es una función

En muchos primeros programas en C aparece una estructura como la siguiente:

```
#include <stdio.h>  /* Incluye una biblioteca de entrada y salida. */

int main(void) {    /* Funcion principal: aqui comienza la ejecucion del programa. */
    printf("Hola desde C.\n"); /* Muestra un mensaje en la consola. */

    return 0; /* Informa que el programa finalizo correctamente. */
} /* Fin de la funcion main. */
```

Figura 1.1: Estructura mínima de un programa en C con `main`.

Aunque a veces se mira como una parte obligatoria del programa, `main` debe entenderse como una función. Tiene nombre, puede tener parámetros, ejecuta instrucciones y devuelve un resultado. Lo especial es su papel: en un programa C usual, `main` actúa como el punto de entrada.

La línea:

```
int main(void)
```

puede leerse por partes:

- `int`: indica que la función devuelve un valor entero;
- `main`: es el nombre especial de la función principal;
- `void`: indica que esta versión de `main` no recibe argumentos por medio de sus parámetros.

El significado de `return 0` en `main`

Cuando `main` termina, puede entregar un valor entero al entorno que ejecutó el programa. En un computador personal, ese entorno puede ser la terminal, el sistema operativo o una herramienta que llamó al ejecutable.

Por convención, y de acuerdo con el comportamiento normal de los programas en C, el valor 0 indica terminación exitosa. Un valor diferente de cero suele usarse para señalar que ocurrió una situación no exitosa, como un error de entrada, una reserva de memoria fallida o una condición que impidió continuar normalmente. La biblioteca estándar también define macros como `EXIT_SUCCESS` y `EXIT_FAILURE`, usadas precisamente para comunicar éxito o falla al finalizar un programa [?].

Comparación

Terminación exitosa: `return 0`; comunica que el programa llegó al final esperado.

Terminación no exitosa: `return 1`; u otro valor diferente de cero suele emplearse para indicar que se presentó un problema. El significado exacto de cada código puede depender del programa y del entorno donde se ejecute.

Esta idea será útil más adelante. Por ejemplo, si `malloc` no puede reservar memoria, el programa puede imprimir un mensaje y terminar con `return 1`; para señalar que no terminó exitosamente.

Comparación con el modelo de ejecución en Arduino

Cuando se programa en Arduino, normalmente no se escribe directamente una función `main`. En cambio, el programa se organiza alrededor de dos funciones conocidas:

```
void setup(void) { /* Se ejecuta una sola vez al iniciar la tarjeta. */  
    /* Configuraciones iniciales: pines, comunicacion serial, sensores, etc. */  
}  
  
void loop(void) { /* Se ejecuta repetidamente despues de setup. */  
    /*Codigo principal que se repite mientras la tarjeta este funcionando. */  
}
```

Figura 1.2: Estructura usual de un programa en Arduino.

En Arduino, `setup` se usa para las configuraciones iniciales y `loop` se ejecuta una y otra vez. La documentación oficial de Arduino describe precisamente que `loop()` se ejecuta de forma consecutiva, permitiendo que el programa cambie y responda continuamente [?].

Esto no significa que la idea de `main` desaparezca. Más bien, el entorno de Arduino oculta esa parte para facilitar la programación. En el núcleo oficial de Arduino AVR, por ejemplo, existe un archivo `main.cpp` que llama a `setup()` y luego ejecuta `loop()` dentro de un ciclo permanente [?].

De manera conceptual, puede representarse así:

```
int main(void) { /* Punto de entrada conceptual del programa. */  
    inicializarHardware(); /* Preparacion interna de la tarjeta. */  
  
    setup(); /* Llamada una sola vez a la configuracion inicial. */  
  
    while (1) { /* Ciclo infinito propio de muchos sistemas embebidos. */  
        loop(); /* Llamada repetida al codigo principal del usuario. */  
    }  
  
    return 0; /* En una tarjeta embebida normalmente no se espera llegar aqui. */  
}
```

Figura 1.3: Representación conceptual simplificada del flujo de Arduino.

La comparación permite ver una diferencia importante. En un programa C ejecutado en un computador, el programa suele iniciar en `main`, realizar una tarea y finalizar entregando un código de retorno. En una tarjeta Arduino, en cambio, el programa normalmente queda ejecutándose de manera continua, porque el microcontrolador debe permanecer atento a sensores, entradas, salidas o comunicaciones.

Comparación

C en computador: el programa inicia en `main`, ejecuta instrucciones, puede llamar funciones y normalmente termina con un código de retorno.

Arduino: el entorno prepara la tarjeta, ejecuta `setup` una vez y luego ejecuta `loop` repetidamente. Por eso se parece más a un sistema que queda funcionando de manera permanente.

1.5. Variables locales y memoria durante la ejecución

Variables locales: qué son y por qué importan

Una **variable local** es una variable declarada dentro de una función o dentro de un bloque de código. Se llama local porque su uso normal queda limitado al lugar donde fue declarada. En el siguiente ejemplo, las variables `a`, `b` y `suma` son variables locales de `main`.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */

int main(void) { /* Inicia la funcion principal del programa. */
    int a = 10; /* Variable local de main. */
    int b = 20; /* Variable local de main. */
    int suma = 0; /* Variable local de main. */

    suma = a + b; /* Usa las variables locales para calcular un resultado. */

    printf("suma = %d\n", suma); /* Muestra el valor guardado en suma. */

    return 0; /* Finalizacion exitosa del programa. */
} /* Al terminar main, sus variables locales normales dejan de ser validas. */
```

Figura 1.4: Variables locales declaradas dentro de `main`.

Desde el punto de vista de la memoria, una variable local normal existe mientras se está ejecutando la función o bloque donde fue declarada. Cuando la función termina, esa variable deja de ser válida. Esta idea será esencial al estudiar punteros, porque no es correcto devolver la dirección de una variable local normal: al terminar la función, esa dirección deja de representar un dato válido para el programa.

Memoria

En este documento, cuando se diga **variable local normal**, se hará referencia a una variable declarada dentro de una función o bloque, cuyo tiempo de vida está asociado a la ejecución de esa función o bloque. Por eso, su dirección no debe usarse después de que esa función haya terminado.

1.6. Programa procedural, ejecución secuencial y memoria

Después del esquema anterior, conviene conectar la organización general del computador con la forma en que se escribe un programa en C. C se usa con frecuencia bajo un enfoque **procedural**: el programa se organiza mediante funciones que reciben datos, ejecutan instrucciones y producen resultados.

También es importante reconocer que, salvo cuando aparecen estructuras de control como condicionales, ciclos o llamadas a funciones, las instrucciones de un programa se ejecutan de manera **secuencial**. Esto significa que el computador atiende una instrucción, luego la siguiente, después la siguiente, y así sucesivamente.

Idea central

En un programa procedural escrito en C, las funciones organizan las tareas y la memoria conserva los datos necesarios para ejecutarlas.

Una primera relación entre código, datos y memoria

En el siguiente programa cada variable declarada ocupa un lugar en memoria, cada asignación cambia el contenido de una variable y cada operación utiliza valores que ya están almacenados.

En el siguiente programa se observa una ejecución secuencial muy sencilla. Primero se declaran tres variables enteras, luego se realiza una operación aritmética y, finalmente, se muestran los resultados en pantalla. Aunque el ejemplo es básico, permite relacionar las instrucciones del programa con la idea de que cada variable ocupa un espacio en memoria.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para usar printf. */

int main(void) { /* Funcion principal: aqui inicia la ejecucion del programa. */
    int a = 10; /* Declara una variable entera llamada a y guarda el valor 10. */
    int b = 20; /* Declara una variable entera llamada b y guarda el valor 20. */
    int suma = 0; /* Declara la variable suma y la inicializa en cero. */

    suma = a + b; /* Suma los valores de a y b, y guarda el resultado en suma. */

    printf("a = %d\n", a); /* Muestra en pantalla el valor almacenado en a. */
    printf("b = %d\n", b); /* Muestra en pantalla el valor almacenado en b. */
    printf("suma = %d\n", suma); /* Muestra en pantalla el resultado de la suma. */

    return 0; /* Indica que el programa finalizo correctamente. */
} /* Fin de la funcion main. */
```

Figura 1.5: Archivo ejemplo_03a_secuencia_datos_memoria.c.

Este ejemplo puede leerse de forma secuencial. Primero se reserva espacio para las variables locales de `main`. Luego se asignan los valores iniciales de `a`, `b` y `suma`. Después se calcula la

suma y se guarda el resultado en la variable **suma**. Finalmente, se muestran los valores en la consola.

La Figura 1.3 resume esta relación entre la ejecución secuencial del programa y la memoria local de **main**. En el lado izquierdo se muestran, de manera simplificada, las acciones principales que realiza el programa. En el lado derecho se representa la memoria local asociada con las variables declaradas dentro de **main**.

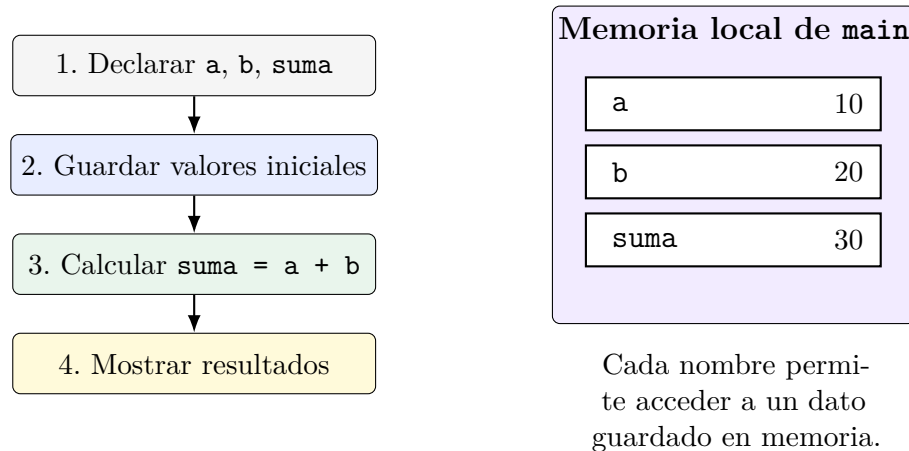


Figura 1.3: Relación entre la ejecución secuencial del programa y la memoria local de **main**.

En la Figura 1.3, la columna de la izquierda representa el avance secuencial del programa. Primero se declaran las variables, luego se les asignan valores, después se realiza la operación aritmética y finalmente se muestran los resultados.

El bloque de la derecha representa la memoria local de **main**. Allí aparecen las variables **a**, **b** y **suma**, junto con los valores que tienen después de ejecutar la operación **suma = a + b**. La figura no pretende mostrar direcciones reales de memoria; su intención es resaltar que cada variable declarada dentro de **main** ocupa un espacio asociado con un dato.

Esta representación ayuda a conectar dos ideas: por un lado, el programa avanza instrucción por instrucción; por otro, las variables conservan en memoria los datos necesarios para que esas instrucciones puedan ejecutarse.

Memoria

Los nombres **a**, **b** y **suma** son identificadores usados por el programador. Durante la ejecución, esos datos ocupan posiciones de memoria. En los primeros programas se trabaja con los nombres de las variables; con punteros, además de los nombres, se empieza a trabajar con sus direcciones.

Funciones como procedimientos que trabajan sobre datos

El enfoque procedural se vuelve más claro cuando el programa se divide en funciones. Cada función concentra una tarea concreta. La Figura 1.4 representa, de manera conceptual, esta

relación entre la memoria local de `main` y la memoria local de `calcularSuma`. La llamada a `calcularSuma(a, b)` permite observar una idea central del enfoque procedural en C: cada función trabaja con sus propios parámetros y variables locales. Aunque `main` y `calcularSuma` participan en una misma tarea, no comparten exactamente las mismas variables. En lugar de eso, la función llamada recibe copias de los valores enviados y produce un resultado que luego retorna.

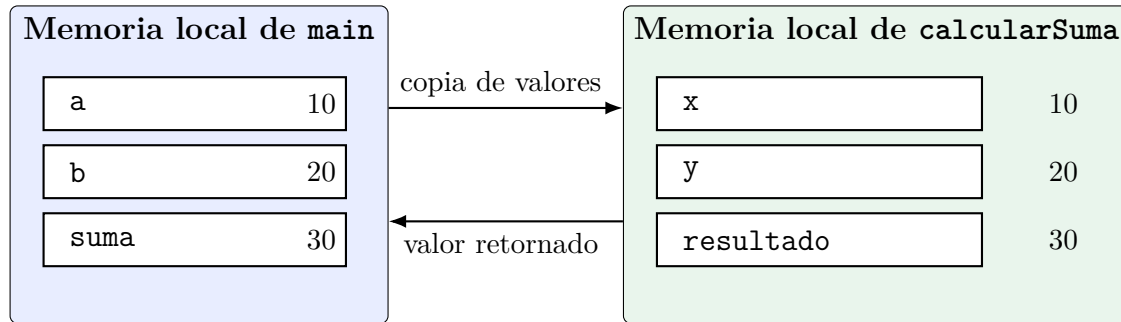


Figura 1.4: Relación entre la memoria local de `main` y la memoria local de `calcularSuma`. Los valores `a` y `b` se copian en los parámetros `x` y `y`, y el resultado calculado se retorna a `main`.

En la Figura 1.4, el bloque de la izquierda representa la memoria local de la función `main`. Allí aparecen las variables `a`, `b` y `suma`. El bloque de la derecha representa la memoria local de la función `calcularSuma`, donde aparecen los parámetros `x` y `y`, además de la variable local `resultado`.

La flecha superior indica que, al llamar `calcularSuma(a, b)`, los valores de `a` y `b` se copian en `x` y `y`. Esto significa que la función trabaja inicialmente con copias de los datos enviados desde `main`. La flecha inferior indica que, una vez realizada la operación, la función devuelve el valor de `resultado`, el cual puede ser almacenado posteriormente en la variable `suma` de `main`.

La figura ayuda a comprender una idea importante: aunque las funciones colaboran entre sí, cada una posee su propio espacio de memoria local. Esta separación será fundamental más adelante, cuando se estudie por qué una función no modifica directamente las variables originales a menos que reciba sus direcciones.

Clave de lectura

En una llamada de función, los parámetros permiten entregar datos a la función. En C, cuando se envían variables normales, se entregan copias de sus valores. Por eso, una función puede calcular con esos datos, pero no modifica directamente las variables originales de `main` a menos que reciba sus direcciones.